



Test Driven Development en Java

Jour 3 — Code Hérité, FitNesse & Outils CI

Formation professionnelle · 3 jours · Niveau intermédiaire



Programme — Jour 3

01 Rappels Jour 2 & introduction

03 Cycle d'évolution du code hérité

05 Tests fonctionnels avec FIT

07 FitNesse — Écriture de tables

09 Outils Open Source & Commerciaux

11 Jenkins & GitHub Actions

13 Travaux pratiques & Bilan final

02 Qu'est-ce que le code hérité ?

04 Techniques de mise sous test

06 FitNesse — Présentation & Architecture

08 FitNesse & TDD — Synergie

10 Intégration Continue — Concepts

12 Couverture de code — JaCoCo & SonarQube

| 01 · Rappels Jour 2 & Introduction

Consolidation et cap sur le Jour 3

Rappels du Jour 2

- Les 5 doubles de test : Dummy, Stub, Fake, Spy, Mock
- Mock vs Stub : comportement vs état
- Doubles manuels sans bibliothèque
- Mockito : `@Mock` , `@InjectMocks` , `@ExtendWith`
- `when().thenReturn()` et variantes
- `verify()` , `times()` , `never()` , `inOrder()`
- **ArgumentCaptor** pour inspecter les arguments
- **Fixtures** : Builder, Object Mother, `@BeforeEach`
- Qualité des tests : lisibilité, isolation, responsabilité
- Styles Classique vs London

Objectifs du Jour 3

- Comprendre et **caractériser** le code hérité
- Appliquer des **techniques de mise sous test** du legacy code
- Utiliser **FIT et FitNesse** pour les tests fonctionnels
- Comprendre la **communication MOA/MOE** via FitNesse
- Mettre en place une chaîne d'**intégration continue**
- Utiliser **JaCoCo** et **SonarQube** pour la couverture
- **Choisir les bons outils** selon le contexte projet

| 02 · Qu'est-ce que le Code Hérité ?

Définir et caractériser le legacy code

Définition — Code Hérité

"Code hérité est tout code sans tests."

— Michael Feathers, *Working Effectively with Legacy Code* (2004)

Définition élargie

- Code écrit **sans tests automatisés**
- Code **difficile à comprendre** ou à modifier
- Code dépendant d'une **technologie obsolète**
- Code dont l'**auteur original** est parti
- Code que **personne n'ose toucher**

Symptômes du code hérité

Signes caractéristiques

- **Peur du changement** : "si je touche ça, je casse tout"
- **Effet domino** : corriger un bug en crée trois autres
- **Tests absents** ou désactivés (`@Ignore` , `// TODO`)
- **Classes gigantesques** : God Class de 5000 lignes
- **Couplage fort** : impossible de tester une classe seule
- **Nommage cryptique** : variables `x` , `tmp` , `data2`



Le coût du code hérité

Impact	Conséquence
Temps de développement	×2 à ×5 plus long
Risque de régression	Très élevé
Onboarding	Semaines au lieu de jours
Moral de l'équipe	Très dégradé
Qualité des nouvelles features	Limitée par la dette
Coût de maintenance	60-80% du budget

La dette technique s'accumule silencieusement jusqu'à paralyser le projet

Les causes du code hérité

- Absence de tests dès l'origine
 - Pression calendaire : "on testera plus tard"
 - Changement d'équipe sans transfert de connaissance
 - Absence de revue de code
 - Conception initiale incorrecte
- Refactoring jamais fait
 - Technologies obsolètes non migrées
 - Documentation insuffisante
 - Couplage excessif entre composants
 - Dépendances non gérées (versions figées)

● Code Legacy — Exemple typique

```
// Code legacy réel – 0 test, 0 documentation
public class Process {
    static Connection c;
    public static void doIt(String s, int x, Object o) {
        try {
            Statement st = c.createStatement();
            ResultSet rs = st.executeQuery(
                "SELECT * FROM STUFF WHERE id=" + x); // injection SQL !
            while(rs.next()) {
                if(rs.getString(1).equals(s)) {
                    // logique métier mélangée avec SQL
                    ((MyInterface)o).exec(rs.getInt(2) * 1.15); // 1.15 = ?
                }
            }
        } catch(Exception e) { e.printStackTrace(); } // silence !
    }
}
```

✓ Code après refactoring TDD

```
// Après refactoring guidé par les tests
public class FactureService {
    private static final double TAUX_TVA_STANDARD = 0.20;
    private final FactureRepository repository;
    private final TVACalculateur tvaCalculateur;

    public FactureTTC calculerFacture(Long factureId) {
        Facture facture = repository.findById(factureId)
            .orElseThrow(() -> new FactureNotFoundException(factureId));
        double montantTVA = tvaCalculateur.calculer(
            facture.getMontantHT(), TAUX_TVA_STANDARD);
        return new FactureTTC(facture, montantTVA);
    }
}
```

| 03 · Cycle d'Évolution du Code Hérité

Reprendre le contrôle progressivement

Le processus de traitement du legacy

- 1 **Identifier** la zone à modifier (feature request ou bug)
- 2 **Caractériser** le comportement actuel (avant tout changement)
- 3 **Mettre sous test** la zone identifiée (filet de sécurité)
- 4 **Refactorer** pour rendre le code testable
- 5 **Ajouter** les tests de la nouvelle fonctionnalité
- 6 **Implémenter** en TDD classique

Étape 1 — Identifier et comprendre

Avant de toucher quoi que ce soit

- Lire le code existant (même sans le comprendre entièrement)
- Tracer l'exécution en mode debug
- Documenter ce qu'on observe (flux de données, dépendances)
- Cartographier les dépendances de la classe

```
# Outil : jdeps pour visualiser les dépendances  
jdeps --print-module-deps MonApplication.jar
```

Étape 2 — Tests de caractérisation

*Les tests de caractérisation décrivent le comportement **actuel** du code, même s'il est incorrect, pour servir de filet de sécurité pendant le refactoring.*

```
// Test de caractérisation – documenter le comportement existant
@Test
void caracterisation_calculDoIt_comportementActuel() {
    // On teste ce que le code FAIT réellement (même si c'est "faux")
    // L'objectif n'est PAS de vérifier la correction logique
    // mais de capturer le comportement pour détecter les régressions
    String resultat = legacy.doIt("test", 42, mock);
    assertEquals("EXPECTED_CURRENT_OUTPUT", resultat);
    // Ce test échouera si quelqu'un change le comportement involontairement
}
```


⚡ Étape 3 — Techniques de mise sous test

Le problème principal

Le code legacy est souvent **non testable** directement car :

- Dépendances instanciées directement dans le code (`new`)
- Méthodes `static` partout
- `Singletons` masquant les dépendances
- Accès direct à la base de données, au réseau, au fichier système

Technique 1 — Extract and Override

```
// Code legacy original – impossible à tester
public class ReportGenerator {
    public void generate(String type) {
        // Dépendance cachée !
        DatabaseConnector db = new DatabaseConnector("jdbc:...");
        List<Data> data = db.fetchData(type);
        // ... traitement ...
    }
}

// Refactoring : Extract and Override
public class ReportGenerator {
    protected DatabaseConnector createConnector() { // ← extraquée
        return new DatabaseConnector("jdbc:...");
    }
    public void generate(String type) {
        DatabaseConnector db = createConnector(); // ← injectable via override
        List<Data> data = db.fetchData(type);
    }
}

// Dans le test : sous-classer et surcharger
class TestableReportGenerator extends ReportGenerator {
    @Override
    protected DatabaseConnector createConnector() {
        return mockConnector; // ← injection du mock
    }
}
```

Technique 2 — Parameterize Constructor

```
// ✗ Avant – dépendance cachée
public class OrderProcessor {
    private final PaymentGateway gateway = new StripeGateway(); // dur
    private final EmailSender sender = new SmtEmailSender(); // dur

    public void process(Order order) { ... }
}

// ✔ Après – injection par constructeur
public class OrderProcessor {
    private final PaymentGateway gateway;
    private final EmailSender sender;

    // Nouveau constructeur pour la testabilité
    public OrderProcessor(PaymentGateway gateway, EmailSender sender) {
        this.gateway = gateway;
        this.sender = sender;
    }

    // Conserver l'ancien pour la compatibilité (pas casser l'existant)
    public OrderProcessor() {
        this(new StripeGateway(), new SmtEmailSender());
    }
}
```

Technique 3 — Break Static Dependencies

```
// ✗ Appel statique – impossible à mocker sans PowerMock
public class UserService {
    public User getUser(Long id) {
        Connection conn = DatabasePool.getConnection(); // statique !
        return UserDao.findById(conn, id);             // statique !
    }
}

// ✓ Wrapper de l'accès statique
public class DatabasePoolWrapper {
    public Connection getConnection() {
        return DatabasePool.getConnection(); // délégation à l'appel statique
    }
}

// ✓ Injection du wrapper dans UserService
public class UserService {
    private final DatabasePoolWrapper pool;
    private final UserDao userDao;

    public UserService(DatabasePoolWrapper pool, UserDao userDao) {
        this.pool = pool;
        this.userDao = userDao;
    }
}
```

Technique 4 — Sprout Method / Class

```
// Code legacy existant – on n'y touche presque pas
public class Invoice {
    public double calculateTotal() {
        // ... 200 lignes de code non testable ...
        return total;
    }
}

// Sprout Method : ajouter une NOUVELLE méthode testable
// sans toucher à l'ancienne
public class Invoice {
    public double calculateTotal() {
        // ... code legacy intact ...
        double baseTotal = calculateBaseTotal(); // ← appelle la nouvelle méthode
        return applyDiscounts(baseTotal);        // ← nouvelle méthode testable
    }

    // Nouvelle méthode – peut être testée en TDD
    double applyDiscounts(double amount) {
        if (amount > 1000.0) return amount * 0.90;
        if (amount > 500.0) return amount * 0.95;
        return amount;
    }
}
```

Technique 5 — Wrap Method

```
// Code legacy – méthode qu'on ne peut pas modifier directement
public class TransactionProcessor {
    public void process(Transaction t) {
        // ... code legacy complexe ...
    }
}

// Wrap : créer une sous-classe qui enveloppe
public class LoggingTransactionProcessor extends TransactionProcessor {
    private final Logger logger;

    @Override
    public void process(Transaction t) {
        logger.info("Processing: " + t.getId());
        super.process(t); // délégation au code legacy
        logger.info("Processed: " + t.getId());
    }
}
```

Choisir la bonne technique

Matrice de décision — Techniques Legacy

Problème	Technique recommandée
<code>new</code> dans le code	Parameterize Constructor
Méthode <code>static</code>	Wrapper + injection
Singleton	Rendre configurable ou wrapper
Grosse méthode monolithique	Sprout Method
Comportement à ajouter	Sprout Class
Modifier sans casser	Wrap Method / Class
Classe non-testable	Extract and Override

💡 Règle d'or : le moins de changement possible pour rendre testable, puis TDD pour le reste.

⚠ Pièges à éviter avec le Legacy

- **Over-refactoring** : trop changer en une fois = risque élevé
- **Tests sans filet** : refactorer avant d'avoir des tests = danger
- **Big Rewrite** : réécrire from scratch est rarement la bonne solution
- **Ignorer les tests de caractérisation** : ils sont indispensables
- **Toucher ce qui n'est pas demandé** : scope creep dans le refactoring

"If it ain't broke, don't fix it." — Mais si ça doit changer : d'abord tester.

TP 6 — Code Hérité

Objectif

Mettre sous test un module legacy fourni par le formateur

Code legacy fourni

- Classe `LegacyFacturationService` (300 lignes, 0 test)
- Dépendances directes : BDD, fichier système, logger statique
- Méthodes statiques, singletons, initialisation dans le code

Étapes

1. Écrire des tests de caractérisation
2. Appliquer **Parameterize Constructor**
3. Extraire les dépendances via interfaces
4. Écrire des tests **TDD** pour une nouvelle règle métier

| 04 · Tests Fonctionnels avec FIT

Framework for Integrated Tests

FIT — Présentation

Qu'est-ce que FIT ?

*FIT (Framework for Integrated Tests) est un outil de tests fonctionnels qui permet aux **non-développeurs** (MOA, testeurs, clients) d'écrire des tests sous forme de tables dans des documents HTML ou Word.*

- Créé par **Ward Cunningham** (co-auteur du manifeste Agile)
- Permet la **collaboration MOA ↔ MOE**
- Les tables = spécifications **executables**
- Disponible en Java, C#, Python, Ruby

Le problème que FIT résout

Fossé communication MOA/MOE

MOA (Maîtrise d'Ouvrage)

"La remise s'applique
quand la commande
dépasse un certain
seuil."

MOE (Maîtrise d'Œuvre)

"Quand montant > 1000 ?"
"À partir de 1001 ?"
"Inclus ou exclus ?"
"Quel taux de remise ?"
"Pour tous les clients ?"

La solution FIT

Une table de données partagée entre MOA et MOE qui s'exécute comme un test et prouve que le logiciel correspond aux attentes.

FIT — Types de tables

ColumnFixture — Tester des calculs

montantHT	taux	tva?	ttc?
100	0.20	20	120
500	0.10	50	550
0	0.20	0	0

ActionFixture — Tester des flux

start		ConnexionFixture	
press		seConnecter	
check		estConnecte	true

ColumnFixture — Implémentation Java

```
// La table FIT est reliée à cette classe Java
public class CalculTVAFixture extends ColumnFixture {

    // Colonnes d'entrée (nommées comme les en-têtes de la table)
    public double montantHT;
    public double taux;

    // Colonnes de sortie (méthode avec "?")
    public double tva() {
        CalculateurTVA calc = new CalculateurTVA();
        return calc.calculerTVA(montantHT, taux);
    }

    public double ttc() {
        return montantHT + tva();
    }
}
```

RowFixture — Tester des collections

```
// Table FIT pour tester une liste de résultats
public class UtilisateursActifsFixture extends RowFixture {

    @Override
    public Object[] query() throws Exception {
        // Retourne les données réelles du système
        UserService service = new UserService(new DatabaseRepo());
        return service.findActifs().toArray();
    }

    @Override
    public Class getTargetClass() {
        return User.class; // FIT mappe les colonnes aux champs
    }
}
```

| 05 · FitNesse — Présentation & Architecture

Wiki de tests collaboratif

FitNesse — Qu'est-ce que c'est ?

FitNesse est un wiki embarqué qui héberge des tables FIT et les exécute comme des tests automatisés, facilitant la collaboration entre toutes les parties prenantes.

Architecture

Navigateur Web



FitNesse Wiki ↔ Pages de tests (tables)



FIT Runner



Code Java (Fixtures)



Application sous test

✦ FitNesse — Fonctionnalités clés

- **Wiki intégré** : pages modifiables via navigateur
- **Exécution à la demande** : bouton "Test" sur chaque page
- **Rapport visuel** : vert/rouge/jaune par cellule
- **Hierarchie de suites** : SuiteTests → GroupeTests → Test
- **Variables** : réutilisables entre pages
- **Includes** : partager des configurations communes
- **Versionning** : historique des modifications
- **Sécurité** : accès en lecture/écriture contrôlé
- **Intégration CI** : REST API pour déclencher depuis Jenkins
- **Multi-fixtures** : FIT, Slim, FitSharp

Installation FitNesse

```
# Télécharger le JAR standalone
wget http://fitnesse.org/fitnesse-standalone.jar

# Lancer le serveur (port 8080 par défaut)
java -jar fitnesse-standalone.jar -p 8080

# Accéder via navigateur
# http://localhost:8080
```

Structure du projet

```
fitnesse/
├── FitNesseRoot/
│   ├── FrontPage/      ← Page d'accueil
│   ├── MaSuite/        ← Suite de tests
│   │   ├── TestCalculTVA/ ← Test spécifique
│   │   └── SetUp/      ← Configuration commune
│   └── plugins.properties
└──
```



Page FitNesse — Syntaxe SLIM

```
!define TEST_SYSTEM {slim}
```

```
!path target/classes
```

```
!path target/dependency/*.jar
```

```
!import
```

```
com.exemple.formation.fixtures
```

```
! | ColumnFixture : CalculTVA
```

montantHT	taux	tva?	ttc?
-----------	------	------	------

100	0.20	20	120
-----	------	----	-----

500	0.10	50	550
-----	------	----	-----

0	0.20	0	0
---	------	---	---

-1	0.20	error	error
----	------	-------	-------

SLIM Fixture — Implémentation

```
// Fixture SLIM (plus moderne que FIT)
public class CalculTVAFixture {
    private double montantHT;
    private double taux;
    private CalculateurTVA calculateur = new CalculateurTVA();

    // Setters pour les colonnes d'entrée
    public void setMontantHT(double montantHT) { this.montantHT = montantHT; }
    public void setTaux(double taux) { this.taux = taux; }

    // Méthodes pour les colonnes de résultat (?)
    public double tva() {
        return calculateur.calculerTVA(montantHT, taux);
    }

    public double ttc() {
        return montantHT + tva();
    }
}
```

Résultat d'exécution

Test Results:

montantHT	taux	tva?		ttc?	
100	0.20	✓	20	✓	120
500	0.10	✓	50	✓	550
0	0.20	✓	0	✓	0
-1	0.20	✗	0	✗	-1

3 tests pass, 1 test fail

Suites de tests FitNesse

```
!contents -R2 -g -p -f -h
```

```
Suite : Facturation
```

— TestCalculTVA	→	✓	Tests de calcul TVA
— TestRemises	→	✓	Tests de remises
— TestGenererFacture	→	✗	Test échoué
— TestExportPDF	→	⚠	Test ignoré

Exécution d'une suite

```
# Via REST API (pour CI)
curl "http://localhost:8080/MaSuite?suite&format=junit"
```

```
# Via Maven plugin
mvn fitnesse:run -Dfitnesse.suite=MaSuite
```

FitNesse — Scenario Tables

```
|scenario|retirer|montant|d'un compte de|solde|donne|restant|
|$montant=|$solde=|
|set solde|@solde|
|retirer|@montant|
|check solde|@restant|
```

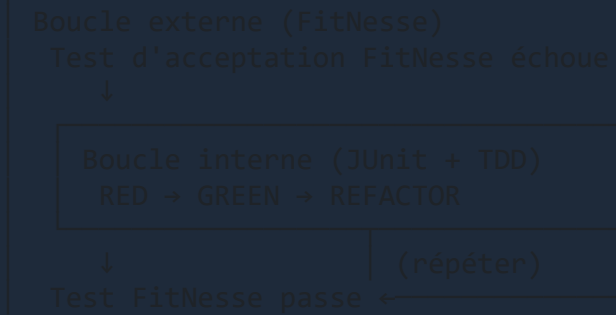
```
||Script : CompteBancaire| | | | |
|retirer|200|d'un compte de|500|donne|300|
|retirer|100|d'un compte de|100|donne|0|
|retirer|300|d'un compte de|100|donne|error|
```


| 06 · FitNesse & TDD — Synergie

Les tests fonctionnels et unitaires se complètent

FitNesse dans le cycle TDD

Double boucle de feedback



Collaboration MOA / MOE avec FitNesse

Processus

1. MOA rédige les cas de test en langage naturel dans le wiki
2. MOE crée les fixtures Java correspondantes
3. MOA vérifie visuellement les résultats (vert/rouge)
4. MOE implémente le code jusqu'à obtenir du vert
5. CI exécute FitNesse à chaque commit

Bénéfices

- Spécifications **executables** et toujours à jour
- Dialogue continu et précis MOA ↔ MOE
- Pas de divergence entre spécification et implémentation

VS Tests FitNesse vs Tests JUnit

Critère	FitNesse	JUnit
Auteur	MOA / testeur	Développeur
Format	Tables wiki	Code Java
Niveau	Fonctionnel	Unitaire/intégration
Vitesse	Lent	Rapide
Isolation	Faible	Forte
Documentation	✓ Naturelle	✗ Code
Refactoring	⚠ Fragile	✓ Robuste

TP 7 — FitNesse

Objectif

Installer FitNesse et créer des tests fonctionnels pour une application de gestion de stock

Étapes

1. Installer FitNesse (`java -jar fitnesses-standalone.jar`)
2. Créer une suite de tests `GestionStock`
3. Écrire des tables pour : ajouter produit, retirer stock, calculer valeur
4. Implémenter les **Fixtures Java** correspondantes
5. Exécuter et observer le rapport vert/rouge

TP 7 — Tables à créer

```
! | ColumnFixture : StockFixture |
| produit | quantite | prix | valeur? |
| Widget  | 10       | 5.00 | 50.00    |
| Gadget  | 3        | 29.99| 89.97    |
| Outil   | 0        | 15.00| 0.00     |

! | Script : GestionStockFixture |
| ajouter produit | Widget | quantite | 5 |
| check stock    | Widget | 15       |
| retirer        | Widget | quantite | 3 |
| check stock    | Widget | 12       |
| retirer        | Widget | quantite | 20 |
| check exception | StockInsuffisantException |
```

| 07 · Les Outils — Vue d'Ensemble

Open Source & Commerciaux

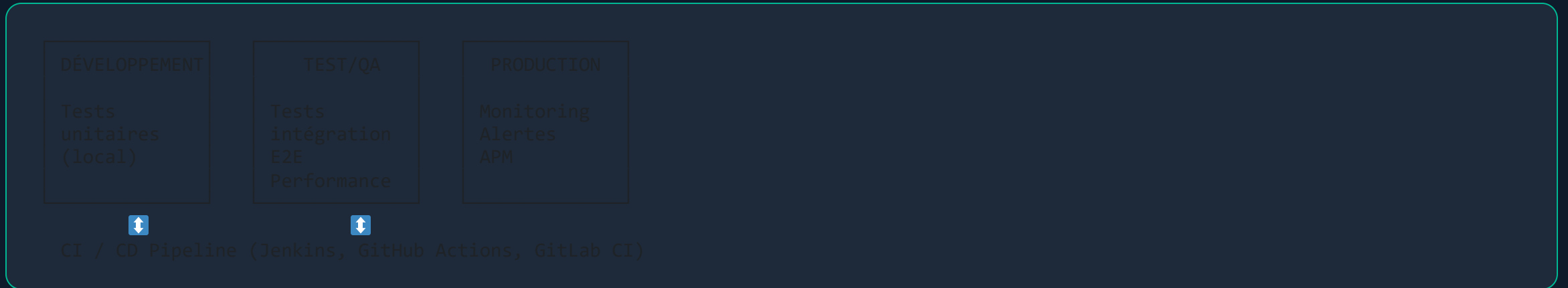
Écosystème des outils de tests Java

- **Frameworks de tests unitaires**
 - JUnit 5, TestNG, Spock
- **Mocking**
 - Mockito, EasyMock, PowerMock, WireMock
- **Assertions**
 - AssertJ, Hamcrest, Truth
- **Tests fonctionnels**
 - FitNesse, Cucumber, JBehave, Serenity
- **Tests d'intégration**
 - Testcontainers, Arquillian, Spring Test
- **Tests de performance**
 - JMeter, Gatling, k6, JMH
- **Tests UI / E2E**
 - Selenium, Playwright, Cypress

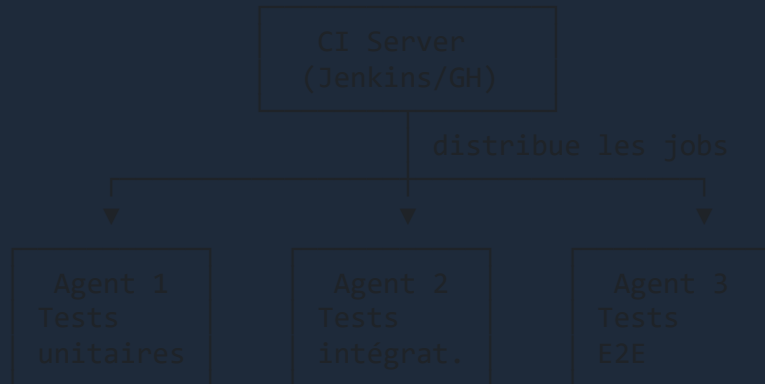


Architecture matérielle de tests

Environnements typiques



Architecture CI — Agents de test



Outils Open Source

- **JUnit 5** — Tests unitaires
- **Mockito** — Mocking
- **AssertJ** — Assertions fluides
- **JaCoCo** — Couverture de code
- **PIT** — Mutation testing
- **Jenkins** — CI/CD server
- **SonarQube** — Qualité de code
- **Testcontainers** — Tests avec Docker
- **Gatling** — Tests de performance
- **FitNesse** — Tests fonctionnels

Outils Commerciaux

Outil	Domaine	Éditeur
Parasoft	Tests unitaires + analyse statique	Parasoft
SmartBear	Tests API, UI	SmartBear
SonarCloud	Qualité code (Cloud)	Sonar
Datadog	Monitoring + tests	Datadog
BrowserStack	Tests cross-browser	BrowserStack
Tricentis Tosca	Tests no-code	Tricentis
Micro Focus UFT	Tests UI automatisés	OpenText

Critères de choix d'un outil

Questions à se poser

- **Communauté** : actif, maintenu, documenté ?
- **Intégration** : compatible avec notre stack (Maven/Gradle, Spring) ?
- **Courbe d'apprentissage** : maîtrisable par l'équipe ?
- **Coût** : open source ? licence par utilisateur ? par build ?
- **Support** : forums, Stack Overflow, documentation ?
- **CI/CD** : s'intègre avec Jenkins/GitHub Actions ?

| 08 · Intégration Continue — Concepts

Automatiser la qualité

Qu'est-ce que l'Intégration Continue ?

*"L'intégration continue est une pratique de développement où les membres d'une équipe intègrent leur travail fréquemment — au moins une fois par jour — et chaque intégration est vérifiée par un **build automatisé** (incluant les tests)."*
— Martin Fowler

Objectif

Détecter les problèmes d'intégration le plus tôt possible

Les principes du CI

- 1 Maintenir un dépôt de code source unique (Git, SVN)
- 2 Automatiser le build (Maven, Gradle)
- 3 Rendre le build auto-testant (tests lancés automatiquement)
- 4 Commiter fréquemment (au moins une fois par jour)
- 5 Ne jamais laisser le build cassé (fix immédiat)
- 6 Construire à chaque commit (déclenchement automatique)
- 7 Garder le build rapide (feedback en < 10 minutes)



Bénéfices du CI

Problème sans CI	Solution avec CI
Intégration douloureuse en fin de sprint	Intégration continue, petits lots
"Ça marchait sur ma machine"	Build reproductible sur serveur neutre
Bugs découverts tard (en recette)	Détection immédiate à chaque commit
Couverture de test inconnue	Rapport JaCoCo automatique
Qualité de code subjective	SonarQube analyse objective
Déploiement manuel et risqué	Déploiement automatisé (CD)

● La règle du build cassé

"Réparer un build cassé est la priorité absolue de l'équipe. Rien d'autre ne passe avant."

Comportements attendus

- **Notification immédiate** en cas d'échec (email, Slack)
- Développeur concerné : **stop work** → fix du build
- Si non réparable rapidement : **revert du commit**
- **Aucun nouveau commit** sur une branche dont le build est rouge
- Objectif : build vert en **moins de 10 minutes**



Pipeline CI typique

```
graph TD;
    A[Commit] --> B[Checkout du code];
    B --> C[Compilation (mvn compile)];
    C --> D["Tests unitaires (mvn test) ← rapides, < 2 min"];
    D --> E[Analyse de code (SonarQube)];
    E --> F["Tests d'intégration (mvn verify) ← < 5 min"];
    F --> G[Package (mvn package)];
    G --> H["Tests E2E (Selenium/Playwright) ← < 10 min"];
    H --> I[Déploiement staging (CD)];
    I --> J[Tests de performance (Gatling)];
    J --> K[Déploiement production];
```

Commit
↓
Checkout du code
↓
Compilation (mvn compile)
↓
Tests unitaires (mvn test) ← rapides, < 2 min
↓
Analyse de code (SonarQube)
↓
Tests d'intégration (mvn verify) ← < 5 min
↓
Package (mvn package)
↓
Tests E2E (Selenium/Playwright) ← < 10 min
↓
Déploiement staging (CD)
↓
Tests de performance (Gatling)
↓
Déploiement production

| 09 · Jenkins & GitHub Actions

Les intégrateurs continus

Jenkins — Présentation

Jenkins, c'est quoi ?

- Serveur d'intégration continue open source le plus utilisé
- Plus de 1800 plugins disponibles
- Supporte tous les langages, toutes les plateformes
- Pipeline as Code : pipeline décrit en Groovy (Jenkinsfile)
- Modèle maître/agents : scale horizontalement

⚙️ Jenkinsfile — Pipeline Java/Maven

```
pipeline {
  agent any

  tools {
    maven 'Maven-3.9'
    jdk 'JDK-17'
  }

  stages {
    stage('Build') {
      steps {
        sh 'mvn clean compile'
      }
    }
    stage('Tests Unitaires') {
      steps {
        sh 'mvn test'
      }
      post {
        always {
          junit 'target/surefire-reports/*.xml'
          jacoco execPattern: 'target/jacoco.exec'
        }
      }
    }
    stage('Tests Integration') {
      steps {
        sh 'mvn verify -P integration-tests'
      }
    }
    stage('Analyse SonarQube') {
      steps {
        withSonarQubeEnv('SonarQube') {
          sh 'mvn sonar:sonar'
        }
      }
    }
    stage('Package') {
      steps {
        sh 'mvn package -DskipTests'
      }
      post {
        success {
          archiveArtifacts artifacts: 'target/*.jar'
        }
      }
    }
  }

  post {
    failure {
      emailExt subject: "Build FAILED: ${env.JOB_NAME}",
              body: "Build #${env.BUILD_NUMBER} failed."
    }
  }
}
```

GitHub Actions — Présentation

GitHub Actions, c'est quoi ?

- CI/CD intégré à **GitHub** (aucun serveur à gérer)
- **YAML-based** : workflows déclarés dans `.github/workflows/`
- **Marketplace** : milliers d'actions pré-construites
- **Runners** : GitHub héberge les machines, ou auto-hébergement
- **Triggers** : push, PR, schedule, manuel, etc.

GitHub Actions — Workflow Java/Maven

```
# .github/workflows/ci.yml
name: CI Java TDD

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'
          cache: maven

      - name: Tests unitaires
        run: mvn test --batch-mode

      - name: Publier rapport JUnit
        uses: dorny/test-reporter@v1
        if: always()
        with:
          name: JUnit Tests
          path: target/surefire-reports/*.xml
          reporter: java-junit

      - name: Rapport couverture JaCoCo
```


⚙️ GitHub Actions — Étapes supplémentaires

```
- name: Analyse SonarCloud
  uses: SonarSource/sonarcloud-github-action@master
  env:
    GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
    SONAR_TOKEN: ${ secrets.SONAR_TOKEN }

- name: Quality Gate Check
  uses: sonarqube-quality-gate-action@master
  timeout-minutes: 5
  env:
    SONAR_TOKEN: ${ secrets.SONAR_TOKEN }

- name: Build et package
  if: github.ref == 'refs/heads/main'
  run: mvn package -DskipTests

- name: Upload artifact
  if: github.ref == 'refs/heads/main'
  uses: actions/upload-artifact@v4
  with:
    name: application-jar
    path: target/*.jar
```

VS Jenkins vs GitHub Actions

Critère	Jenkins	GitHub Actions
Hébergement	Auto-hébergé	Cloud (GitHub)
Configuration	Groovy (Jenkinsfile)	YAML
Coût	Gratuit + infra	Gratuit (limites)
Maintenance	✓ Votre équipe	✗ Pas de serveur
Plugins	★★★★★ (1800+)	★★★★★ (marketplace)
Intégration Git	✓ Plugin	✓ Natif
Scalabilité	✓ Agents	✓ Runners
Maturité	★★★★★	★★★★★

GitLab CI — Alternative

```
# .gitlab-ci.yml
stages:
  - test
  - quality
  - build

tests-unitaires:
  stage: test
  image: maven:3.9-eclipse-temurin-17
  script:
    - mvn test
  artifacts:
    reports:
      junit: target/surefire-reports/*.xml
    paths:
      - target/site/jacoco/

sonarqube:
  stage: quality
  script:
    - mvn sonar:sonar -Dsonar.host.url=$SONAR_URL
  only:
    - main
```

| 10 · Couverture de Code

JaCoCo & SonarQube



Qu'est-ce que la couverture de code ?

Définition

*La couverture de code mesure le **pourcentage de code source** exercé par l'exécution des tests automatisés.*

Types de couverture

- **Line Coverage** : lignes exécutées / total des lignes
- **Branch Coverage** : branches (if/else, switch) couvertes
- **Method Coverage** : méthodes appelées
- **Class Coverage** : classes instanciées
- **Instruction Coverage** : instructions bytecode exécutées

JaCoCo — Java Code Coverage

Présentation

- Plugin Maven/Gradle le plus utilisé pour la couverture Java
- Intégré avec Jenkins, GitHub Actions, SonarQube
- Rapport HTML visuel par ligne
- Pas d'instrumentation à la compilation (agent Java)

```
<!-- pom.xml - Configuration JaCoCo -->
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.11</version>
  <configuration>
    <excludes>
      <exclude>**/generated/**</exclude>
      <exclude>**/*DTO.class</exclude>
    </excludes>
  </configuration>
  <executions>
    <execution>
      <goals><goal>prepare-agent</goal></goals>
    </execution>
    <execution>
      <id>report</id>
      <phase>test</phase>
      <goals><goal>report</goal></goals>
    </execution>
  </executions>
</plugin>
```

Rapport JaCoCo — Lecture

Légende des couleurs dans le rapport HTML

-  Vert — Code couvert par les tests
-  Rouge — Code non couvert
-  Jaune — Branch partiellement couverte (1 branche sur 2)

Exemple de rapport

Class	Line%	Branch%	Methods%	
CalculateurTVA	100%	100%	100%	✓
VirementService	85%	75%	90%	⚠
LegacyReportGen	12%	8%	15%	✗

Seuils de couverture recommandés

Par type de code

Type de code	Seuil recommandé
Logique métier critique	> 90%
Services applicatifs	> 80%
Contrôleurs REST	> 70%
Getters/Setters auto-générés	Exclure
Code généré (Lombok, JAXB)	Exclure
Configuration Spring	Exclure

 *La couverture ne garantit pas la qualité des tests — un test sans assertion peut avoir 100% de couverture*

SonarQube — Qualité de code globale

SonarQube, c'est quoi ?

- Plateforme de **qualité de code** continue
- Analyse : bugs, vulnérabilités, code smells, dette technique
- Support de **30+ langages** dont Java
- **Quality Gate** : bloque le déploiement si seuils non atteints
- Versions : Community (gratuit), Developer, Enterprise

Bugs

Problèmes de code qui peuvent causer des comportements erronés en production

Vulnérabilités

Faibles de sécurité potentielles (injection SQL, XSS, etc.)

Code Smells

Code maintenable mais qui pourrait être amélioré (dette technique)

Security Hotspots

Code à réviser manuellement pour valider la sécurité

⚙️ SonarQube — Configuration Maven

```
<!-- pom.xml -->
<properties>
  <sonar.host.url>http://localhost:9000</sonar.host.url>
  <sonar.login>${env.SONAR_TOKEN}</sonar.login>
  <sonar.coverage.jacoco.xmlReportPaths>
    target/site/jacoco/jacoco.xml
  </sonar.coverage.jacoco.xmlReportPaths>
  <sonar.exclusions>
    **/generated/**,
    **/*DTO.java,
    **/config/**
  </sonar.exclusions>
</properties>
```

```
# Lancer l'analyse
mvn clean verify sonar:sonar
```

Quality Gate — SonarQube

Quality Gate par défaut (Sonar Way)

- ✓ Nouveau code doit avoir :
 - Couverture > 80%
 - 0 bug critique
 - 0 vulnérabilité critique
 - < 3% de code dupliqué
 - Rating maintenabilité : A
 - Rating sécurité : A

✗ Si non respecté → Déploiement BLOQUÉ

SonarQube — Exemples d'issues détectées

```
// 🐛 Bug détecté : NullPointerException potentielle
public String getUsername(Long userId) {
    User user = userRepo.findById(userId); // peut retourner null !
    return user.getName(); // NullPointerException si user == null
}

// 🔒 Vulnérabilité : injection SQL
String query = "SELECT * FROM users WHERE id = " + id; // ❌

// 🧴 Code Smell : méthode trop complexe (complexité > 10)
public void process(Order order) {
    if (...) { if (...) { for (...) { if (...) { ... } } } }
}

// 🔒 Security Hotspot : données sensibles en log
log.info("User password: " + password); // ❌ à réviser
```

Architecture complète — CI avec qualité

```
Developer commit
  ↓
GitHub / GitLab
  ↓
CI Pipeline (Jenkins / GitHub Actions)
  ↓
mvn clean verify    ← Tests unitaires + intégration
  ↓
JaCoCo coverage     ← Rapport de couverture
  ↓
SonarQube analysis  ← Qualité de code
  ↓
Quality Gate ?
  ✓ PASS → Deploy staging
  ✗ FAIL → Notification équipe + build bloqué
```

TP 8 — Pipeline CI Complet

Objectif

Mettre en place une chaîne CI complète pour le projet TDD

Étapes

1. Configurer JaCoCo dans `pom.xml` avec seuil > 80%
2. Lancer `mvn verify` et analyser le rapport HTML
3. Installer SonarQube (Docker) ou utiliser SonarCloud
4. Créer un Jenkinsfile (ou workflow GitHub Actions)
5. Déclencher un build et observer le pipeline
6. Faire échouer un test et observer la notification

TP 8 — SonarQube avec Docker

```
# Lancer SonarQube en local avec Docker
docker run -d \
  --name sonarqube \
  -p 9000:9000 \
  sonarqube:community

# Attendre le démarrage (~1 minute)
# Accéder : http://localhost:9000
# Login : admin / admin

# Analyser le projet Maven
mvn clean verify sonar:sonar \
  -Dsonar.host.url=http://localhost:9000 \
  -Dsonar.login=TON_TOKEN_SONAR
```


TP 9 — FitNesse + CI

Objectif

Intégrer FitNesse dans le pipeline CI

Étapes

1. Créer une suite FitNesse pour le projet de stock (TP 7)
2. Configurer le **plugin Maven FitNesse**
3. Ajouter l'exécution FitNesse dans le Jenkinsfile
4. Configurer la **publication du rapport** FitNesse dans Jenkins
5. Faire échouer un test FitNesse et observer le pipeline

```
<plugin>  
  <groupId>uk.co.javaahelp.fitnesses</groupId>  
  <artifactId>fitnesses-launcher-maven-plugin</artifactId>  
  <version>1.4.2</version>  
</plugin>
```

| 11 · Étude et Choix d'un Intégrateur Continu

Prendre la bonne décision

Critères de choix d'un CI

Critères techniques

- **Écosystème** : compatible avec Git, Maven, Docker ?
- **Pipeline as Code** : versionning des pipelines
- **Parallélisation** : plusieurs builds en parallèle ?
- **Agents distribués** : scale horizontal ?
- **Intégrations** : SonarQube, Slack, Jira, Artifactory ?

Critères organisationnels

- **Coût total** : licence + infrastructure + maintenance
- **Compétences** de l'équipe (groovy vs yaml vs autre)
- **Support** : community vs entreprise
- **Hébergement** : on-premise vs cloud

Comparatif CI — Tableau de décision

Comparatif des serveurs CI

Critère	Jenkins	GitHub Actions	GitLab CI	CircleCI
Hébergement	Auto	Cloud	Auto/Cloud	Cloud
Config	Groovy	YAML	YAML	YAML
Plugins	★★★★★	★★★★★	★★★★★	★★★★
Coût	Gratuit	Gratuit*	Gratuit*	Freemium
Maturité	15 ans	5 ans	10 ans	10 ans
Difficulté	Moyenne	Faible	Faible	Faible

Recommandations selon le contexte

Startup / Petit projet

→ **GitHub Actions** : zéro infrastructure, rapide à configurer, intégré à GitHub

Entreprise / Besoin de contrôle

→ **Jenkins** : sur site, personnalisable à l'infini, plugins exhaustifs

Projet GitLab

→ **GitLab CI** : intégration native, pipeline as code natif

Besoin de performance extrême

→ **CircleCI** ou **GitHub Actions** avec runners parallèles

Étude — Outil de couverture de test

Outils disponibles

- **JaCoCo** ← Standard Java, gratuit, intégration SonarQube
- **Clover** ← Commercial, Atlassian (arrêté)
- **Cobertura** ← Obsolète (plus maintenu)
- **OpenClover** ← Fork open source de Clover

Recommandation 2026

JaCoCo est le standard incontestable pour Java.

Couplé à SonarQube, il couvre tous les besoins de couverture et de qualité.

| 12 · Bilan Final — Les 3 Jours

Récapitulatif complet de la formation

Jour 1 — Ce que nous avons appris

Fondamentaux TDD

- Cycle Red → Green → Refactor et son rythme
- Patron AAA / Given-When-Then
- Tests FIRST : Fast, Independent, Repeatable, Self-Validating, Timely
- Gestion des exceptions : `assertThrows()`
- Refactoring sécurisé et conception émergente

JUnit 5

- Annotations, assertions, tests paramétrés
- `@Nested`, `@Tag`, `@Timeout`
- AssertJ pour les assertions fluides

Jour 2 — Ce que nous avons appris

Doubles de test

- 5 types : Dummy, Stub, Fake, Spy, Mock
- Implémentation **manuelle** vs bibliothèques

Mockito

- `@Mock` , `@InjectMocks` , `@Spy`
- `when/thenReturn` , `verify` , `ArgumentCaptor`
- Vérification d'ordre avec `InOrder`

Qualité des tests

- Fixtures, Builder Pattern, Object Mother
- Anti-patterns : logique dans les tests, sur-spécification
- Styles **Classique** vs **London**

Jour 3 — Ce que nous avons appris

Code hérité

- Définition, symptômes, coûts
- Tests de caractérisation
- Techniques : Extract & Override, Parameterize Constructor, Sprout

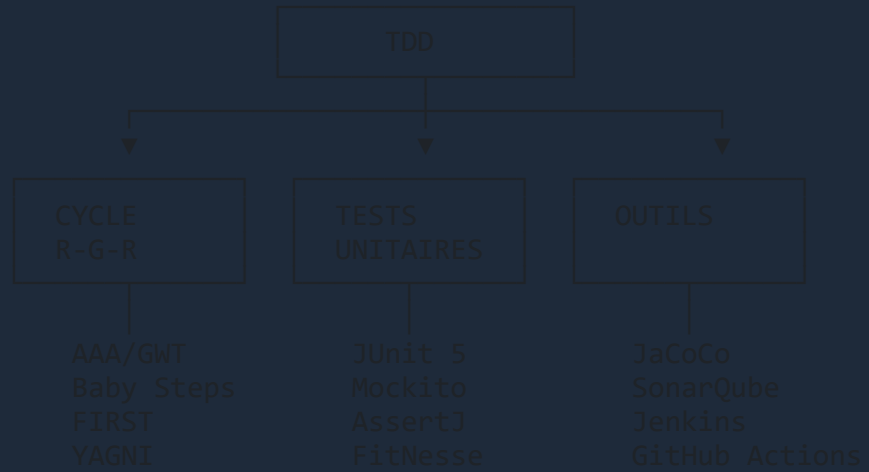
FitNesse

- Tests fonctionnels collaboratifs MOA/MOE
- Tables FIT / SLIM, suites, execution CI

Outillage

- JaCoCo : couverture de code
- SonarQube : qualité globale et Quality Gate
- Jenkins et GitHub Actions : CI/CD

La carte mentale TDD



Prochaines étapes — Plan d'action

Court terme (1-2 semaines)

- Appliquer TDD sur le prochain ticket/feature
- Mettre en place JaCoCo sur votre projet actuel
- Viser 70% de couverture sur le nouveau code

Moyen terme (1-3 mois)

- Mettre sous test une zone de code legacy critique
- Configurer SonarQube et définir un Quality Gate
- Partager les pratiques TDD avec l'équipe

Prochaines étapes — Avancé

Long terme (3-12 mois)

- Mettre en place FitNesse pour la collaboration MOA/MOE
- Atteindre 80%+ de couverture sur l'ensemble du projet
- Former l'équipe (pair programming TDD)
- Intégrer Mutation Testing (PIT) dans le pipeline CI

Lectures recommandées

- *Test-Driven Development by Example* — Kent Beck
- *Working Effectively with Legacy Code* — Michael Feathers
- *Growing Object-Oriented Software Guided by Tests* — Freeman & Pryce

TP 10 — Projet Final Intégré

Objectif

Réaliser un projet complet de bout en bout avec toutes les pratiques vues

Scénario : Système de Gestion d'une Bibliothèque

Développer en TDD complet :

- Gestion des livres (CRUD)
- Système d'emprunt et retour
- Gestion des amendes pour retard
- Notifications aux membres

TP 10 — Exigences techniques

Stack à utiliser

- JUnit 5 + AssertJ pour les tests unitaires
- Mockito pour les doubles de test
- H2 pour les tests d'intégration
- JaCoCo avec seuil > 85%
- FitNesse pour 2 règles métier clés
- GitHub Actions pour le pipeline CI
- SonarQube (Docker) pour la quality gate

Livrable

Dépôt Git avec pipeline CI vert + rapport couverture

TP 10 — Tests attendus (extrait)

```
// Tests unitaires (JUnit 5 + Mockito)
void emprunter_LivreDisponible_MarqueLivreEmprunte()
void emprunter_LivreDejaEmprunte_LeveException()
void retourner_AvecRetard_CalculeAmende()
void notifier_RetourProche_EnvoieRappel()
```

```
// Tables FitNesse
```

CalculAmende	
joursRetard	amende?
0	0.00
1	0.50
7	3.50
30	15.00
31	20.00 ← plafond



Checklist — Bonnes pratiques TDD

À vérifier sur votre code

- [] Tests écrits **avant** le code de production (TDD pur)
- [] Nommage des tests : `methode_Contexte_Resultat`
- [] Chaque test vérifie **une seule chose**
- [] Tests **FIRST** : rapides, indépendants, répétables
- [] Pas de logique (`if/for`) dans les tests
- [] Mocks uniquement pour les **dépendances externes**
- [] **Fixtures** extraites dans `@BeforeEach` ou builders
- [] Couverture **JaCoCo** > 80%
- [] Pipeline **CI** vert en permanence
- [] **Quality Gate** SonarQube respecté

Les 10 commandements du TDD

- Écrire le **test avant** le code de production
- Un **seul test** qui échoue à la fois
- **Minimum de code** pour faire passer le test
- **Refactorer** après chaque test qui passe
- **Nommer** les tests clairement et expressément
- **Isoler** chaque test de ses dépendances
- **Vérifier** une seule chose par test
- **Committer** souvent avec les tests verts
- **Mesurer** la couverture régulièrement
- **Améliorer** la qualité des tests comme du code

Citations inspirantes

"Écrire des tests n'est pas la preuve qu'on n'a pas confiance en soi. C'est la preuve qu'on est professionnel." — Robert C. Martin

"Le code qui n'est pas testé est du code bogué." — Jeff Atwood

"Chaque fois que vous avez la tentation de taper quelque chose dans une REPL, écrivez-le comme test à la place." — Martin Fowler

"Les bons tests sont la meilleure documentation que vous puissiez avoir." — Michael Feathers

Ressources complètes

Livres incontournables

- *Test-Driven Development by Example* — Kent Beck (2002)
- *Working Effectively with Legacy Code* — Michael Feathers (2004)
- *Growing Object-Oriented Software Guided by Tests* — Freeman & Pryce (2009)
- *xUnit Test Patterns* — Gerard Meszaros (2007)
- *Clean Code* — Robert C. Martin (2008)
- *Refactoring* (2e édition) — Martin Fowler (2018)

Ressources en ligne

- **JUnit 5** : junit.org/junit5/docs
- **Mockito** : site.mockito.org
- **AssertJ** : assertj.github.io
- **JaCoCo** : jacoco.org
- **SonarQube** : docs.sonarqube.org
- **FitNesse** : fitnesse.org
- **Jenkins** : jenkins.io/doc
- **GitHub Actions** : docs.github.com/actions
- **PIT Mutation** : pitest.org
- **Testcontainers** : testcontainers.com



QCM Final — Jour 3

Question 1

Quelle est la définition du code hérité selon Michael Feathers ?

- A) Code écrit il y a plus de 5 ans
- B) Code écrit dans une technologie obsolète
- C) **Code sans tests automatisés** ✓
- D) Code écrit par une équipe externe

Question 2

Qu'est-ce qu'un test de caractérisation ?

- A) Un test qui vérifie que le code est correct
- B) **Un test qui documente le comportement actuel du code** ✓
- C) Un test de performance
- D) Un test de sécurité

Question 3

Quel outil FIT/FitNesse sert à la collaboration MOA/MOE ?

- A) JUnit 5
- B) Mockito
- C) **FitNesse — wiki de tests fonctionnels** ✓
- D) SonarQube

Question 4

Que mesure la couverture de branche (branch coverage) ?

- A) Le nombre de lignes exécutées
- B) Le nombre de méthodes appelées
- C) Le **pourcentage de branches if/else couvertes** 
- D) Le nombre de classes testées

Question 5

Qu'est-ce qu'un Quality Gate SonarQube ?

- A) Une interface graphique de SonarQube
- B) **Un ensemble de seuils de qualité bloquant le déploiement si non respectés** 
- C) Un plugin Maven pour la couverture
- D) Un rapport HTML de couverture

Évaluation de la formation

Votre retour est précieux !

- **Contenu** : les sujets couverts répondaient à vos attentes ?
- **Rythme** : trop rapide, trop lent, adapté ?
- **TPs** : trop difficiles, trop simples, pertinents ?
- **Formateur** : disponibilité, pédagogie, expertise ?
- **Ce qui vous a le plus apporté** : quel sujet ?
- **Ce qui manquait** : qu'auriez-vous voulu aborder ?

Compétences acquises

Vous êtes maintenant capables de...

-  Pratiquer le TDD dans vos projets Java quotidiens
-  Écrire des tests unitaires de qualité avec JUnit 5
-  Utiliser Mockito efficacement pour isoler les dépendances
-  Approcher le code hérité avec méthode et prudence
-  Collaborer avec la MOA via FitNesse
-  Mettre en place un pipeline CI avec tests automatisés
-  Mesurer et améliorer la couverture de code
-  Analyser la qualité du code avec SonarQube

Félicitations !

Vous avez terminé la formation Test Driven Development en Java !

Merci pour votre engagement et votre participation active

Continuez à pratiquer — le TDD s'apprend par la pratique quotidienne

Formation TDD Java · © 2026 · Bonne continuation !



Questions & Échanges Finaux

Posez toutes vos dernières questions !

Certificats de formation disponibles sur demande